南昌大学超算俱乐部考核报告

姓名：周家名 学号：9110125032 进阶方向：C++（cpp） GitHub 仓库链接：_____

基础阶段

一、Git 部分

技术报告

差不多写了几道 Git 相关的关卡：我大概分为三部分吧，一条命令拆解的「基础操作」关（初始化，分清工作区和暂存区，留下第一个版本），以及两道实操关「找回历史中的内容」「让两条历史汇合」。下面按顺序记录每关的做法和思路。

第一部分：Git 基础操作

这一部分要求掌握三类最常用的查询命令：看工作区状态、看提交历史、读历史版本里的文件。

```
git status --short --branch
```

刚接触时我只会敲 `git status`，默认输出会弹出一大段文字，很多提示看不懂。后来加上 `--short --branch` 之后清爽多了：

- `git status` 是状态查询的基础命令，默认输出完整说明和操作指引；
- `--short` 切成精简模式，只显示「状态缩写 + 文件名」；
- `--branch` 在输出最前面加上当前分支名和分支状态。

输出第一行是 `## 分支名`，后面的每行开头一个字母表示状态：`M` 已修改、`A` 新增、`D` 已删除。日常用得最多的其实是这个精简版，扫一眼就知道仓库现在什么情况。

```
git log -2 --oneline
```

`git log` 默认会把每条提交的哈希、作者、时间、说明全都打出来，满屏看得眼花。加上参数之后：

- `-2` 限制只显示最近 2 条；
- `--oneline` 每条提交压缩成一行，只留短哈希 + 提交说明。

每行就是「短哈希 + 提交说明」，哈希值是每次提交的唯一标识，后面回退版本、对比差异、看文件都要靠它。

`git show HEAD:notes.txt`

这条是最难绕明白的。语法上冒号前面是版本标识（提交哈希、分支名、`HEAD` 都可以），冒号后面是仓库里的文件路径，作用是不切版本也能读到历史版本里某个文件的内容。

关键是理解它读的是**已提交到版本库的快照**，不是我正在编辑的工作区文件。我当时本地改了 `notes.txt` 没提交，跑这条命令输出的还是上次提交的内容，困惑了好一阵才想通。

第二部分：找回历史中的内容

题目：仓库里有两个提交 c0（notes.txt 是 version one）、c1（version two），工作区还有未提交的临时修改。要求取回 c0 里 notes.txt 的内容，同时**完整保留提交历史、不动 HEAD、不把修改放进暂存区**。

我的做法：读完题先排除了会动 HEAD 的 `git reset`、会切换提交的 `git checkout c0`，最后选了 `git restore --source` 这条路，只覆盖工作区，HEAD、暂存区、历史全都不受影响，正好卡在题目要求上。

核心就一条命令：

```
git restore --source=c0 -- notes.txt
```

以 c0 为来源，把 notes.txt 恢复到对应版本，只作用于工作区。

做完用下面几条验证状态：

```
cat notes.txt          # 内容是 version one，和 c0 一致
git diff -- notes.txt  # 工作区已经恢复成旧版本
git diff --cached      # 暂存区为空，没有误 add
git log -2 --oneline   # 历史没动，HEAD 还指向 c1
```

这里补一句：这题其实有第二种解法，先清掉工作区改动，再用 `git revert c1` 生成一个新提交把内容倒回 version one。两种都能过，我选了不动历史、只恢复工作区的那条，更贴合「保留历史 + 轻量修改」。

第三部分：让两条历史汇合

题目：main 和 topic 从 c0 处分叉，各自改完提交后，把 topic 合并进 main，最终生成一个带两个父节点的合并提交，两个分支的文件都保留，topic 指针还在，工作区干净。

我的做法：

```
git branch topic                        # 基于 c0 建 topic 分支
printf 'main ready\n' > main.txt
git add -- main.txt
git commit -m "learn: prepare main"     # main 分支先提交

git switch topic
printf 'topic ready\n' > topic.txt
git add -- topic.txt
git commit -m "learn: prepare topic"    # topic 分支提交

git switch main
git merge --no-edit topic               # 把 topic 合进 main
```

两个坑，都是我自己踩出来的：

1. **合并方向搞反了**。合并永远是「把别的分支合到你现在站着的分支」，必须先 `switch main` 再 merge。我一开始没切就直接 merge，结果把 main 合进了 topic，完全反了。
2. **没加 `--no-edit` 会弹 vim**。做题环境是非交互的，必须加这个参数跳过编辑提交信息，不然卡在 vim 里半天出不来。

验证用 `git log --graph --oneline --all` 能直接看到「分叉—合并」的树形结构，第一次直观看到合并提交有两个父节点，之前只在AI对话框上见过这个概念。

感悟报告（Git）

说实话在这之前我几乎没碰过 Git，只知道 add、commit、push 这几个词，当时也基本没用过，也忘了啥时候了解的了，对工作区、暂存区、提交这些底层概念一直是模模糊糊的。这次写这些题，又拷打好久AI，还去找个黑马程序员的网课看了看。

第一个触动是 `git status`。默认命令输出一大段，看着烦，加了 `--short --branch` 之后突然就清爽了。我才意识到参数是用来定制输出的，让命令贴合你自己的需求。一

开始连 M、A、D 这三个字母都分不清，对着说明一个个对，才记牢是「修改、新增、删除」。

`git log` 也一样，不加 `--oneline` 满屏信息，加了之后每条一行。日常大部分时候我只需要知道「改了什么、哈希是多少」，信息太多反而干扰判断。

最难绷的是 `git show HEAD:notes.txt`。我想当然以为它显示的是我正打开的文件，结果我本地改了几行再跑，输出还是老内容，愣了好一会儿。后来去查 HEAD 的含义，翻了工作区和版本库的区别才想通：Git 的每次提交就像一个游戏存档，HEAD 指向最新的那个档，这条命令读的是存档里的文件，不是我正在编辑还没「存」的文件。想通这一点，突然就对「版本控制」有实感了——原来 Git 就是帮我们把每一次提交都存下来，随时能回去读旧版，不怕改坏。

做实操两关的时候，第一反应还是想用 `git reset` 回退到 c0，仔细读题发现要求「保留历史」，才反应过来 reset 会动 HEAD、直接改掉历史，不符合要求。中间还差点顺手 `git add` 把文件暂存了，看到检查步骤要「暂存区为空」才停住，也第一次真切搞懂工作区和暂存区是两个独立的地方。合并那关最深的坑就是方向搞反，试了两次才记住「把别的分支合到你现在站的分支」。

这三关做下来最大的感受是：Git 的命令不是靠背的，每条命令背后都是对仓库某一层的改动。理解工作区、暂存区、提交历史这三层，就能想清楚每条命令会影响什么、不影响什么，也就不怕把仓库搞乱了。之前总觉得 Git 复杂容易出错，现在明白只要逻辑清楚，状态是能精准控制的。

二、Linux 部分

技术报告

Linux 部分我也完成了几道题大差不差也分为三部分吧：第一部分也就是文件检索与程序调用、一道文件字节操作、一道 Bash 脚本（这个我有点不会啊）。整体是从「只会单独敲命令」到「能把一串命令串成完整流程」的过程。

第一部分：密钥检索与文件处理

题目：在 Linux 环境下找到目标密钥 `STAR`，用题给的程序处理输入文件，生成符合格式的结果文件并提交。

整体思路分四步：

1. **环境准备**：观察目录结构，`mkdir input` 建目录，把待处理的源文件 `cp` 进去统一管理。
2. **密钥检索**：`grep "关键词" 文件名` 在文件里搜，定位到密钥 `STAR`。
3. **程序调用**：程序需要带参数调用，参数有 `-e` 和 `-k` 两种。我一开始问AI说用 `-k` 啊，结果发现在这个平台环境里用不了，然后发现能 help，最后还是靠 `-e` 才跑通，全程用的都是 `-e`。
4. **校验提交**：生成输出后检查内容。提交不过时排查出两个典型问题——缺感叹号 `!`、输出文件多了换行。清理多余换行、保证单行、内容完整后再提交。

踩过的坑：

- 不能把目录直接当文件传给程序，会报错；
- 平台校验对格式极其敏感，除了文字内容，换行符、特殊符号都会影响判定，文字对了但换行/符号缺了照样提交失败；
- 参数选择得先自己试：`-k` 在我这个环境里用不了，最后还是靠 `-e` 跑通的，别光看说明就往上套。

第二部分：文件字节操作

题目：按字节要求改输入文件，跑程序拿到输出，提取 flag。难点在于换行符、空格这种看不见的字符会直接改变文件字节长度。

整体思路：

1. 读取原文件，删掉末尾的换行；
2. 追加 1 个空格，再加换行，生成新的输入文件；
3. 跑程序得到 output；
4. 把 output 完整复制到 flag 文件提交；
5. 校验不过时用 `cat -A` 看隐藏换行符。

核心一条命令：

```
printf "%s \n" "$(head -c -1 /workspace/input.txt)" > /workspace/input
```

拆开看：

- `head -c -1` 读文件全部内容、删掉最后 1 个字节（原文件末尾的换行符）；
- `"%s \n"` 里 `%s` 放原文，后面加一个空格再加换行 `\n`；
- `> /workspace/input` 覆盖写回 input 文件。

然后：

```
./exam > output      # 跑程序，结果写进 output
cp output flag       # 完整复制到 flag，避免手打出错
cat -A input         # 查看隐藏字符，检查有没有多余空行
```

三个问题的排查：

- 文件多出一行空行：`echo` 会自动在末尾加换行，很容易多字节，改用 `printf` 精准控制，配合 `cat -A` 确认每个换行、空格的位置；
- 手动输 flag 导致失败：手打容易错、复制粘贴可能带看不见的换行，直接用 `cp output flag` 一字不差地复制；
- 没删原始换行：文本文件末尾默认带换行，这个换行也算 1 字节，不删掉的话追加空格后总字节数就不符合要求，用 `head -c -1` 切掉再重加。

第三部分：Bash 脚本（让命令找到路）

题目：写一个 Bash 脚本，接收参数、做校验、读环境变量、按条件走不同分支、调用工具并返回对应的退出码。我之前完全没学过 Bash，这题也是靠AI写的。

我的做法：

1. 先使用AI解答然后读懂需求：脚本要接参数、校验参数数量、读环境变量、按条件分支、调用工具、返回结果码。
2. 补基础语法（还没补完）：变量定义、`if` 判断、`$1` `$2` 取参数、`exit` 返回退出码，一行行弄懂。
3. 用 `nano route.sh` 编辑，写完 `chmod +x route.sh` 加执行权限。（权限也还没学😔）
4. 分场景测试：模拟缺参数、参数合法、环境变量缺失各种情况，看输出和返回码定位问题。（这个真是全程跟着AI）

脚本骨架大概是：

```
#!/bin/bash
# 用 $# 判断参数数量，不满足就提示并退出
# 读环境变量，判断是否存在
# if-elif 多分支，匹配不同条件调用对应工具，捕获返回码
# 最后 exit 返回状态码，供平台判题读取
```

测试方式：

```
chmod +x route.sh
./route.sh 参数1 参数2
echo $?    # 看上一条命令的退出码，验证返回结果
```

遇到的问题：

- 完全没学过 Bash，看不懂语法，心里发怵。解决：不一次性写一大段，一行行拆，先写最简单的参数判断跑通，再一点一点加逻辑；拿 bash 和 C 对比着理解分支、变量。
- 语法报错：`if` 的空格、`[ ]` 前后必须有空格、少分号、引号不匹配都会报错。每次只加一小段，写完立刻跑，快速定位。
- nano 不会保存退出：`Ctrl+O` 保存、回车确认、`Ctrl+X` 退出。
- 没执行权限报 `permission denied`：`chmod +x route.sh`。
- 分支跑错、返回码不对：分场景单独测，用 `echo $?` 看返回码，找到条件写错的地方。

感悟报告（Linux）

Linux 这几题，是这轮考核里我费劲最多的地方。因为之前没怎么用过命令行环境，我是从「会单独敲命令」到「能把一整套流程串起来」的过程，中间踩的坑基本都是看不见的细节。

第一题做检索的时候，思路特别乱。单独 `mkdir`、`grep` 我都认识，但把建目录、复制文件、查密钥、调程序、校验提交串成一条线，就不知道每步之间怎么衔接了。一开始我问AI去试 `-k`，结果平台环境里 `-k` 用不了，绕了一圈最后还是用 `-e` 跑通的，这是第一个坑。第一次提交失败，发现是少了个感叹号，补上还是不过，当时很疑惑，以为内容对了就能过。排查半天才意识到，平台不光看文字，换行符这种看不见的格式也会影响结果——这是我之前完全没想过的点。

第二题字节操作更典型。一开始我觉得「肉眼看到文字一样就行了」，结果 `echo` 会自动加换行，文件凭空多出空行，校验一直过不了，完全找不到错在哪。靠 `cat -A` 把隐藏字符显示出来，才看清文件里看不见的换行。还有一开始想手动把 output 里的字符串敲进 flag 文件，后来意识到手打必错，直接 `cp` 复制才是稳妥的。这题让我真正明白：Linux 里换行、空格都是实实在在的字节，做字节校验的题不能凭肉眼看，得看文件原始内容。

第三题是我第一次碰 Bash，刚拿到题目完全看不懂，心里很慌，甚至想「能不能直接拿 C 写」。之前只学过 C，Bash 语法风格差太多，一堆特殊符号、参数变量都是陌生

的。我没硬着头皮一次写完，而是选择一小段一小段写、边写边测。过程里踩了一堆小坑：`if` 两边忘空格、不知道 nano 怎么存、脚本忘加执行权限。每次报错我都去分是语法错还是逻辑错，最后还是靠AI大人搞的，我想着这样去写后几道也没啥意义，如果某个点不会去查AI可能会加快学习效率，但像这样完全没接触，倒不如先去补课。

这几题下来我最深的体会是：Linux 实操不是背命令，而是理解每条命令、每个参数到底用在什么场景，还要盯住文件的细节格式。很多次不是命令敲错了，是看不见的换行、空白导致校验失败。遇到报错别盲目改内容，一步步拆：先看报错信息，确认目录文件对不对，再确认参数选没选对，最后查文件内容和格式。这个「分步排查」的习惯，比多背几条命令有用多了。在写 Linux 考核题之前，我仅仅只是知道 Linux 的一些基础命令，写题的时候发现我好像根本没有入门，写完这几道感觉还是比之前好很多吧。

进阶阶段（C++ 方向）

技术报告

进阶我选了 C++ 方向，一共五道题：基础整数统计、设备巡检记录统计、固定容量读数窗口、多文件统计工具、四线程快速排序。这几题难度递进，从单文件编译运行，一路到多文件 CMake 工程、内存错误修复，再到多线程并行。我自己是有点 C++ 基础的，所以这几题本身不算太难，主要的不适应在换到了 Linux 命令行环境——以前都在 IDE 里写完点运行，这次得手动编译、重定向、diff 校验，多文件构建和多线程对我来说也都是全新的东西。

题目 1：基础整数统计

题目：读 n 个整数，输出 count、sum、min、max。

算法本身很简单，真正的考点在 Linux 命令行开发流程：

- 不能依赖 IDE 一键运行，要手动 `g++` 编译成可执行文件；
- 用输入重定向 `<` 从文件读测试用例，而不是键盘输入；
- 用 `diff` 精确对比输出和标准答案，排查肉眼看不到的格式问题；
- 提交文件要符合根目录规则，避免源码缺失类报错。

关键点：

- 最值用第一个输入值初始化 min/max，别初始成 0，否则全负数输入会出错；

- 输出严格 `key=value`，等号两边无空格，每行末尾换行；
- `main` 结尾 `return 0`。

踩坑：一开始直接跑程序终端卡住，误以为死循环，其实是程序默认从标准输入读、必须 `< 样例文件` 重定向；还有肉眼看输出一致但评测报不匹配，用 `diff` 定位到是多了个空格，从那以后养成了「先 diff 再提交」的习惯。

题目 2：设备巡检记录统计

题目：读多条巡检记录，按固定顺序统计 printer、scanner、router、server 四类设备的条数和总运行小时数，输出总小时数最高的设备。

关键点：

- 用结构体给每类设备存「计数 + 总时长」两个字段，初值都 0，天然满足「无记录也输出 0 0」；
- **并列规则**：总小时数相同时，选固定顺序里更靠前的设备，不是先出现的。所以找最大值只用严格 `>`，不用 `>=`，这样数值相同时保留顺序靠前的下标；
- 字符串要完整比对设备类型，别只比首字符。

踩坑：样例里 printer 和 server 总小时数相同，我一开始输出成 server，重新审题 + 看提示才明白固定顺序优先，修正了比较逻辑。

题目 3：固定容量读数窗口（内存错误修复）

题目：只保留最近 8 个读数，修复原代码数组越界写入，支持任意数量输入，正确输出保留个数、最早值、最新值、总和。

关键点：

- 用环形缓冲区（循环数组）：数组大小固定 8，写入位置 `write_pos % CAPACITY`，保证下标始终落在 0~7，从根上解决越界；
- 保留数量 `kept` 最多到 8，再多输入也不再涨；
- **最早值定位**：缓冲区没满时最早值在下标 0；满了之后最早值就是「下一个即将被覆盖的位置」`write_pos % CAPACITY`，不是固定 0；
- **求和**：从 oldest 位置开始环形遍历 kept 个有效元素，别把已经被覆盖的旧数据算进去。

踩坑：原代码直接用循环变量 `i` 当下标，输入超过 8 个就崩，改成环形下标取模修复；还有一开始从下标 0 开始求和，把旧数据也算了，改成从 oldest 开始环形遍

历。

题目 4：多文件统计工具（CMake 构建）

题目：这道题和前面几题不一样，不再是单个 `.cpp` 文件，而是一个拆成三部分的 C++ 工程——入口 `main.cpp`、统计 `stats.h` / `stats.cpp`、格式化 `format.h` / `format.cpp`。初始工程能读，但构建不出来或者输出不对，要求修好工程，让干净构建出来的程序满足输出契约：输入 n 个整数，输出 count、sum、min、max 四行，退出码 0。

要修的两类问题：

1. **构建入口没列全源文件**：`CMakeLists.txt` 里的 `add_executable` 少了实现文件，构建时链接报 `undefined reference`。补齐成三个 `.cpp` 都列进去：

```
add_executable(main main.cpp stats.cpp format.cpp)
```

2. **print_report 声明和定义对不上**：头文件 `format.h` 里声明的参数顺序是 `(count, total, smallest, largest)`，但实现文件里的顺序/类型跟它对不上。C++ 会把不同签名当成重载，于是 `main` 调用的那个签名没有定义、链接失败。以头文件声明和 `main` 的调用语义为准，把实现改一致。

还有一个坑：sum 要用 64 位整数。样例 2 的 sum 是 5000000000，已经超过 32 位有符号整数的上限，所以求和相关的都用 `long long`。

修好的格式化部分：

```
// format.h
void print_report(int count, long long total, int smallest, int largest);

// format.cpp
void print_report(int count, long long total, int smallest, int largest) {
    std::cout << "count=" << count << '\n'
              << "sum=" << total << '\n'
              << "min=" << smallest << '\n'
              << "max=" << largest << '\n';
}
```

构建与验证：

```
cmake -S . -B build
cmake --build build
./build/main < samples/01-wide.in
```

用样例 2（5 个 1000000000，sum 应该输出 5000000000）验证 64 位求和对不对。

题目 5：四线程快速排序实验

题目：用最多 4 个工作线程做并行快速排序，验证正确性，写实验报告分析并行行为。

线程库和数量控制

用的是 C++17 标准库的 `std::thread`，没引入 OpenMP、TBB 这些第三方框架。在此之前我完全不知道线程是什么、线程库是什么，是边问边学才勉强理解的。

数量控制：在并行快排的递归函数里，分区完成后给左半区间开新线程，右半区间当前线程自己处理。用一个全局原子计数器 `active_threads` 统计活跃线程数，严格限制最多 4 个。每次开线程前检查计数，没到上限才开；线程跑完减一。以前我都不会这种计数方式，只会设个普通变量写进循环里。

```
std::atomic<int> active_threads(0);
const int MAX_THREADS = 4;

if (active_threads.load() < MAX_THREADS) {
    active_threads.fetch_add(1);
    std::thread left_thread(parallel_qsort, std::ref(arr), left, mid - 1);
    parallel_qsort(arr, mid + 1, right);  // 当前线程处理右半区
    left_thread.join();                   // 汇合子线程
    active_threads.fetch_sub(1);
}
```

划分与串行截止

分区用 Lomuto 方案，取区间末尾元素当 pivot，遍历区间把小于等于基准的换到左边，最后把基准放到分界位置，返回分界下标：

```
int partition(std::vector<int>& arr, int left, int right) {
    int pivot = arr[right];
    int i = left - 1;
    for (int j = left; j < right; ++j) {
        if (arr[j] <= pivot) {
            ++i;
            std::swap(arr[i], arr[j]);
        }
    }
    std::swap(arr[i + 1], arr[right]);
    return i + 1;
}
```

这段是为数不多我能自己理解并独立写出来的，不容易。

串行截止：区间长度 ≤ 4 时不再递归、不开线程，直接插入排序。避免线程创建和调度的开销大过排序本身。

```
if (right - left + 1 <= SERIAL_THRESHOLD) {
    insertion_sort(arr, left, right);
    return;
}
```

线程汇合

汇合位置在开子线程的同一层递归里：当前线程处理完右区间后 `join()` 阻塞等左子线程跑完，保证整个区间有序后再返回。每一层递归汇合自己开的线程，层层向上，最后全部回收，不存在野线程。

正确性验证：两种方式交叉验证——相邻逆序检查（遍历确认每个相邻元素前小后大）+ 标准库对照（同一份数据用 `std::sort` 排一遍，逐元素对比）。

原始数据 `42 7 19 42 3 88 15 7 61 0 54 23 91 15 36 100`，排完输出 `0 3 7 7 15 15 19 23 36 42 42 54 61 88 91 100`，两种验证都一致。

环境说明：实验跑在单 CPU 核心的沙盒环境，系统靠时间片轮转调度线程，没法真正并行，所以耗时只能用来观察行为，不能当并行加速的性能依据。单核下多线程还要额外付线程创建和切换的开销，耗时甚至可能比单线程还高。

感悟报告（C++）

C++ 这几题对我来说不算陌生，毕竟有点底子。真正让我头疼的不是 C++ 本身，而是换了个环境——以前写 C++ 都是在 IDE 里写完直接点运行，这次要在 Linux 命令行里手动编译、重定向输入、用 diff 校验，这套流程是新的，一开始挺不适应。

前两题还算顺，但也暴露了我不太严谨的地方。整数统计那题，程序跑起来终端卡住，我第一反应是「死循环了」，其实是没做输入重定向、程序在等键盘输入。还有明明肉眼看着输出一模一样，评测就是报不匹配，最后用 diff 才发现多打了个空格。设备统计那题更典型，printer 和 server 总小时数一样，我直接输出成 server，重新审题才知道是「固定顺序里靠前的优先」。这两题让我意识到，比赛/考核里真正的坑往往不在算法，而在审题和格式细节。

第三题修内存越界，我第一次真正体会到「越界」是什么后果——原代码拿循环变量直接当下标，输入一超 8 个就崩。改成环形缓冲区、用取模回绕之后才稳。这里还学到一点：最早值不是永远在下标 0，缓冲区满之后它其实是「下一个要被覆盖的位置」，这个认知是我自己绕了半天才绕明白的。

第四题多文件工程，是我第一次碰 CMake。之前都是单个 .cpp 直接 g++ 编译，这次工程拆成了好几个文件，还得用 CMake 构建，一开始连 CMakeLists.txt 是什么都不知道。好在题目给的两个问题挺典型的——一个漏了源文件、一个函数声明定义对不上，对着报错一个个改，也就慢慢明白多文件工程是怎么组织、怎么链接的了。

多线程那题是全新的东西。老实说，在此之前我完全不知道线程是什么、线程库是什么，问 AI 问了好久才勉强理解，但要我自己独立写还是困难。尤其是线程数量控制，我以前只会设个普通变量手动计数，看了原子计数器 `std::atomic<int>` 的写法才去了解这种安全的计数方式。最后能跑通、能验证正确，是有 AI 帮助、也有不少自己的思考在里面。

这里插一句题外话：这份四线程的报告我其实写了第二遍。第一遍写完去上了个厕所，回来发现容器超时了，而且忘了保存，全没了，只能崩溃着重写第二遍。所以现在看到这份，还挺有感触的。而且后面看群里学长们发的消息，发现我写的还是太简陋了，四线程我就直接开了4个跑了一遍，完全没有想着去试验，开不同数量线程结果对比，也没有那些好看的图（估计就是学长说的领导不喜欢看吧┬┴┬┴＿┬┴┬），虽然考虑了，多看线程理论上会增加效率，但开线程本身也是一种浪费，也许能找到一个最优的点吧（二次函数图像最高点的意思）

这几题做下来，我最大的收获不是又练了几道 C++，而是从「在 IDE 里点运行」过渡到了「在命令行里编译、重定向、diff 校验」这套流程，也慢慢理解了超算里常说的「工程化」——算法写对只是第一步，编译、内存安全、并发、格式规范，每一环都决定程序到底能不能跑对。

写在最后

整场考核写下来，最大的感受就是难，真的难，也让我清楚看到自己要学的还有一大堆。

以前跟着网课学过点 Linux 和 Git 的基础命令，当时觉得"这些我都会了"，结果真做起题来才发现自己只是纸上谈兵，遇到一道完整的题完全不知道从哪儿下手。这段时间几乎每天都在对着 AI 一遍遍问那些很蠢的问题，问完看完答案，很多时候还是一知半解，只能硬着头皮接着磨。

不过磨下来也不是没收获。至少我现在明白，光"看过"、"背过"命令是没用的，真本事是上手写出来的。所以接下来我想把 Linux 和 Git 好好补一补，多上手写点题、做点小项目，把那些"半懂不懂"的地方慢慢变成"真的会"。